



OVERVIEW & QUICKSTART

SigniQ

System overview, pipeline architecture, measured evaluation results and quickstart.

TEAM PROMETHEUSZ · SIH26141

Quantum-secured document signatures · classical software simulation · 2026-10-02

SigniQ

Team Prometheus · SIH26141

Quantum-inspired threat detection for digital-signature security — built as an honest, inspectable software simulation.

SigniQ is a classical software simulation of a quantum-secured document pipeline. It pairs a six-state quantum key distribution model with a teleportation-based quantum digital signature protocol, then layers explicit statistical threat detection on top — no AI, no ML: every accept/reject verdict comes from a closed-form bound or threshold applied to measurement statistics. You can seal a document, attack it four different ways, and watch each forgery get rejected for a named, explainable reason.

The problem statement asks for exactly this shape of work: *"a simulation of a teleportation-based quantum digital signature protocol with a threat-detection layer that, explicitly without any AI or ML, uses quantum principles — Pauli eigenstates, projective measurements and statistical analysis of measurement outcomes — to detect forgery, impersonation, replay attacks and quantum channel manipulation by computing forgery probabilities and verification accuracy from measurement statistics, evaluated through attack simulations that show detection rates and false-accept rates while preserving the protocol's information-theoretic security guarantees."* Every clause of that sentence is implemented and mapped to source in

`docs/DELIVERABLES.md` .

What it is / what it is not

✓ It is	✗ It is not
A classical simulation of QKD and teleportation-based QDS	Quantum hardware, a physical channel, or a real detector
Statistical detection: Hoeffding/Chernoff bounds, binomial tails	An AI/ML system — zero ML dependencies (grep-verifiable)
An educational prototype with reproducible seeded experiments	Production cryptography
A tamper-evident audit ledger proving internal consistency	Publisher authentication — an audit root alone doesn't prove <i>who</i> published

Contents

- Quickstart
- How it works
- Features
- Evaluation results

- The audit ledger
- Limitations
- Repository layout
- Documentation
- Verification
- Deploying
- License & team

SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware

Quickstart

Requirements: **Rust (cargo)** and **Node.js 20+ with npm**.

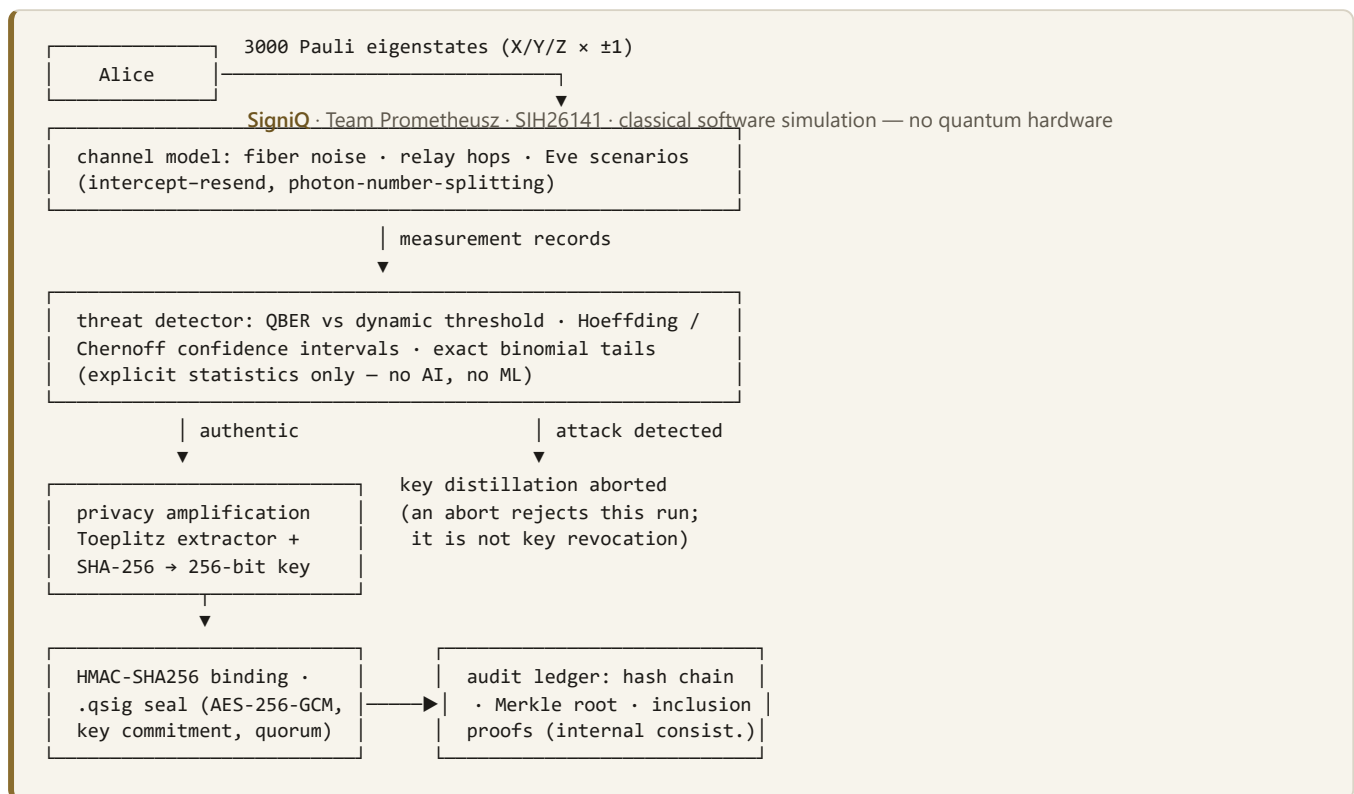
```
cd frontend
npm ci
npm run build
cd ..

# 2. Run the API server (serves the built dashboard too)
cargo run -p server
```

Open the printed URL — normally `http://127.0.0.1:8080`. For frontend hot reload, run `npm run dev` from `frontend/` while the API server runs. A terminal interface exists too: `cargo run -p tui`.

How it works

The whole pipeline in one picture (every block is a classical software model):



1 — Key generation (six-state QKD simulation). Alice prepares qubits in one of six Pauli eigenstates (X/Y/Z × ±1). Bob measures in random bases; the sifted key's error rate (QBER) is compared against a Hoeffding-derived threshold that tightens as sample count grows. A clean channel distills a 256-bit key via privacy amplification; an intercepted channel crosses the threshold and key distillation aborts.

2 — Signatures (teleportation-based QDS). Signatures are built from genuine Bell-measurement outcomes of a statevector teleportation engine — CNOT + Hadamard, Born-rule collapse, Pauli corrections. Forging the whole signature means guessing every Bell outcome: **P(forgery) = $4^{(-q \cdot \lambda)}$** ($\approx 1.5 \times 10^{-5}$ at default parameters $q=8, \lambda=1$). Verdicts follow the protocol's 1-ACC / o-ACC / REJ semantics with transferability, and a temporal trap (single-use nonces + sifting-window sequence) kills replays.

3 — Threat detection (statistics, not models). Detection rates and false-accept rates come from seeded Monte-Carlo attack batteries over a confusion matrix. Forgery, impersonation, replay, channel tampering and unauthorized verification each have a named attack routine and a named rejection reason.

4 — Documents. Files are sealed into .qsig containers (AES-256-GCM, key commitment, Shamir-split quorum optional) and delivered peer-to-peer or through k-of-m consensus rings. The recipient verifies the container cryptographically before trusting a byte of it.

Features

- **Six-state QKD simulation** — Pauli eigenstate preparation, sifting, QBER, finite-sample classification, relay hops and fiber-noise modeling, live channel streaming with attack scenarios.
- **Teleportation-based QDS** — statevector Bell measurement, Born-rule outcomes, Pauli corrections, 1-ACC/o-ACC/REJ verdicts, transferability (second-verifier consensus), replay-state persistence across restarts.

- **Threat-detection layer** — Hoeffding/Chernoff confidence engine (`/api/stats/bounds`), dynamic rejection thresholds, per-attack-class detection rates, explicit false-accept rate.
- **Document vault** — `seal/open` . `sig` containers (v2 payload AES-256-GCM; v3 envelope hides all metadata), tamper battery, attack theater replay.
- **P2P transfer + consensus rings** — laptop-to-laptop delivery, same-server user delivery, k-of-m attestation gates that stay locked below quorum.
- **Audit ledger** — persistent JSONL events, hash chain, Merkle root, inclusion proofs, portable export, offline bundle verification in the browser.
- **Evaluation** — `/api/qds/metrics` returns verification accuracy, per-class detection, false-accept/false-alarm rates, forgery probabilities (empirical + theoretical), and operation timings.
- **Dashboard + TUI** — React dashboard with live SSE monitor, guided run, team dossier; terminal UI via `cargo run -p tui`.

Evaluation results

Measured live on this codebase (isolated scratch instance, Sept 29, 2026; `trials=120, seed=42` — reproducible with the command in [Verification](#)):

Metric	Teleport QDS	Six-state QDS
Verification accuracy	100.0%	100.0%
Detection rate (TPR)	100.0%	100.0%
False-accept rate (missed attacks)	0.0%	0.0%
False-alarm rate (legitimate flagged)	0.0%	0.0%
Per-class detection (forgery / impersonation / replay / channel tampering / unauthorized)	100% each	100% each
Theoretical forgery probability	$4^{-8} \approx 1.53 \times 10^{-5}$	—
Empirical forgery success	0 / 120	0 / 120
Timing (sign / verify)	41 μ s / 18 μ s	machine-specific

End-to-end behavior in the same run: 4/4 tamper modes rejected with named causes; consensus ring locked at k–1, opened at quorum with byte-exact document recovery; 6/6 QDS attack classes rejected; audit chain verified intact over 131 events. These are simulation measurements under seeded test distributions — not universal security guarantees (see [Limitations](#)).

The audit ledger

Every security-relevant action — seal, verify, attack, key derivation, delivery — is appended to a hash-chained JSONL ledger with a Merkle root and per-event inclusion proofs. The dashboard can export a self-contained

bundle and verify it offline; `GET /api/audit/verify` re-checks the chain. **Scope, stated plainly:** this proves the ledger's internal consistency. It does not prove *who* published it — that requires an expected root from an independent trusted source. There is deliberately no key revocation lifecycle; a QKD abort or rejected signature is not revocation.

Limitations

1. **All quantum behavior is classical software simulation.** No quantum hardware, physical detector, or physical channel participates anywhere.
2. **Simulation ≠ certification.** The protocol's information-theoretic arguments are preserved in the model; simulations demonstrate the statistics, they cannot certify physical devices.
3. **The audit ledger proves internal consistency only**, not publisher identity.
4. **No key-revocation lifecycle exists.**
5. **Metrics are seeded-reproducible, not universal** — detection rates describe the implemented test distributions; timings are machine-specific.
6. **Educational prototype** — review cryptographic constructions and deployment configuration before any production use.
7. **Deployment configuration ships in the repo** (Dockerfile, render.yaml — see [Deploying](#)); it is not itself a security review of the hosting setup.

Deploying

The repository is deployment-ready: a multi-stage `Dockerfile` builds the dashboard and the Rust server into one image (persistent state under a `/data` volume), and a Render Blueprint (`render.yaml`) describes the single-service deployment with a `/api/health` check. A one-command local rehearsal:

```
docker build -t sih26141 . && docker run -p 8080:8080 sih26141
```

Server environment variables (PORT, HOST, FRONTEND_DIST, AUDIT_LOG, USERS_FILE, QDS_EVENT_LOG, TRENT_SEED) are documented in `docs/PROJECT_HANDBOOK.md` .

Repository layout

└─ quantum/	SignIQ : Team Prometheus, SIH26141: classical software simulation — no quantum hardware
└─ pa/	six-state QKD model: Pauli states, channel, relays, CHSH
└─ detection/	privacy amplification: Toeplitz extractor, entropy accounting
└─ qds/	Hoeffding/Chernoff bounds, dynamic thresholds
└─ crypto/	teleportation + six-state QDS, attacks, metrics, temporal trap
└─ sealing/	shared primitives
└─ audit/	.qsig containers: AES-256-GCM, commitments, Shamir quorum
└─ attacks/	hash chain, Merkle root, inclusion proofs
└─ server/	QKD-channel attack scenarios
└─ main_app/	Axum API + static dashboard hosting
└─ tui/	CLI demo binary
└─ tests/	terminal UI
└─ frontend/	cross-crate integration tests
	React 19 + TypeScript + Vite dashboard

Documentation

Document	Purpose
docs/DELIVERABLES.md	Clause-by-clause audit against the official problem statement
docs/SIGNIQ_WHITEPAPER.md	Full technical whitepaper: formal models, proofs, evaluation
docs/PROJECT_HANDBOOK.md	Architecture, API reference, data formats, configuration
docs/DASHBOARD_MANUAL.md	Panel-by-panel user manual for the web dashboard
docs/QDS_MATH_MODEL.md	Formal mathematical model of the teleportation QDS
docs/PAPERS.md	Paper → mechanism → code → test literature map
docs/pdf/*.pdf	Branded PDF editions of the four documents (committed) — regenerate with <code>node scripts/build-docs-pdf.mjs</code>

Verification

<pre>cargo test --workspace # 21 test binaries, all pass cd frontend && npm run build && npm run lint # dashboard build + lint # Reproduce the evaluation numbers above against a fresh isolated instance: cargo build -p server PORT=8090 AUDIT_LOG=target/scratch/audit.jsonl QDS_EVENT_LOG=target/scratch/qds.jsonl \ USERS_FILE=target/scratch/users.json target/debug/server.exe & node scripts/smoke-isolated.mjs # 47/47 checks</pre>
--

License & team

Built by **Team Prometheus** for SIH26141. Team member profiles are in the dashboard's dossier (flame mark in the hero). This repository is an educational prototype; no warranty of fitness for any security purpose is given or implied.